

Implementation report: α -attractor inflaton models in COSMOLATTICE

Covers arXiv:2510.18656 (Ellis, Garcia, Olive, Verner)

October 2, 2026

Abstract

This report documents the COSMOLATTICE implementation of every inflationary model found viable in arXiv:2510.18656: the generalized and deformed α -attractor E-models and T-models. It lists the new files, explains how the model headers and the initial-condition helper were built, and justifies every parameter choice in the benchmark parameter files. All families are covered by two model headers, `attractorE.h` and `attractorT.h`, plus 14 parameter files. The physics background (data constraints, derivative formulas, analytic approximations) is in the companion note `notes/inflation_models.tex`.

Contents

1	Summary	1
2	New and modified files	2
2.1	New files	2
2.2	Modified files	2
3	Models implemented	2
4	How the model headers were built	3
4.1	Run-time parameters	3
4.2	Program variables	3
4.3	Numerically safe forms	4
5	The initial-condition helper	4
6	Choice of the parameters	5
6.1	Model parameters (α, k, κ)	5
6.2	Number of e -folds N_*	5
6.3	Lattice and run settings	6
6.4	Benchmark files	6
7	Validation	7
8	How to build and run	7
9	Open points	7

1 Summary

- **Two model headers** cover the four viable families. A deformed model with $\kappa = 1$ is exactly the corresponding generalized model, and $k = 2$, $\alpha = 1$, $\kappa = 1$ of the E-model is Starobinsky.

α , k , κ , λ and the daughter coupling g are read at run time, so no recompilation is needed to scan them.

- **One Python helper** (`notes/attractor_ics.py`) solves the homogeneous background. Given (α, k, κ, N_*) it returns the CMB normalization λ and the inflaton amplitude and velocity at the end of inflation.
- **14 benchmark parameter files** reproduce the points quoted in the paper.
- Both models compile against COSMOLATTICE 2.0 and ran without errors in short tests (Sec. 7).

2 New and modified files

2.1 New files

File	Content
<code>models/attractorE.h</code>	Generalized + deformed E-model (α -Starobinsky), inflaton + daughter field.
<code>models/attractorT.h</code>	Generalized + deformed T-model, inflaton + daughter field.
<code>models/parameter-files/attractorE*.in</code> (8 files)	E-model benchmarks (Table 1).
<code>models/parameter-files/attractorT*.in</code> (6 files)	T-model benchmarks (Table 1).
<code>notes/attractor_ics.py</code>	Background solver: λ , φ_i , $\dot{\varphi}_i$ from (α, k, κ, N_*) .
<code>notes/inflation_models.tex</code> (+ PDF)	Physics note: which models, why, potentials, derivatives, benchmark table.
<code>notes/implementation_report.tex</code> (+ PDF)	This report.

Two build directories, `build_attractorE/` and `build_attractorT/`, were also created. They are ignored by git.

2.2 Modified files

No tracked source file of COSMOLATTICE was modified: the new models are self-contained headers picked up by the existing CMake option `-DMODEL=<name>`.

Before this work the repository was updated to the latest upstream version (COSMOLATTICE 2.0, commit `acc8278d`). Two local changes to tracked files (deletion of `src/models/parameter-files/lphi4.in`, edit of `lphi4U1.in`) were reverted to the upstream versions. They are saved in git `stash` entry “local changes before pulling v2.0”. Your untracked files (`src/models/lphi4_only.h`, `m2phi2.h`, their `.in` files, `papers/`) were not touched. In v2.0 models live in `models/`, not `src/models/`.

3 Models implemented

With $x \equiv \varphi/M_P$, $b \equiv \sqrt{2/(3\alpha)}$, $c \equiv b/2 = 1/\sqrt{6\alpha}$ and $V_0 = \frac{3}{4}\lambda M_P^4$:

$$\text{attractorE: } V = V_0 [\kappa(1 - \cosh bx) + \sinh bx]^k, \quad (1)$$

$$\text{attractorT: } V = V_0 \left[\frac{1}{2}(1 + \kappa + (1 - \kappa) \cosh bx) \right]^2 \tanh^k(cx). \quad (2)$$

These are the deformed models of the paper (its Sec. V.B, “Deformed No-Scale Attractors”). They reduce to

- $\kappa = 1$: generalized E-model $V_0(1 - e^{-bx})^k$ and generalized T-model $V_0 \tanh^k(cx)$ (paper Sec. V.A);
- $\kappa = 1, k = 2$: standard E- and T-models; additionally $\alpha = 1$: Starobinsky.

We checked analytically that the paper’s superpotential construction (its Appendix A) gives the bracket in Eq. (1) raised to the power k . Near the minimum both potentials behave as φ^k for any κ .

A second scalar χ (field index 1) is coupled through $\frac{1}{2}g^2\varphi^2\chi^2$, as in the existing `lphi4.h` and `tanh2.h`. The default is $g = 0$: then χ is a free massless spectator and the run studies the inflaton’s self-resonance only.

4 How the model headers were built

The headers follow the structure of the existing `models/tanh2.h` and `models/lphi4.h`: a `ModelPars` struct (2 scalars, 2 potential terms, `double`), a constructor that reads parameters and sets the program-variable rescaling, and the functions `potentialTerms`, `potDeriv`, `potDeriv2` for each field.

4.1 Run-time parameters

Key	Default	Symbol	Meaning
<code>lambda</code>	required	λ	CMB normalization, $V_0 = \frac{3}{4}\lambda M_P^4$
<code>alphaAtt</code>	1	α	attractor parameter. Not called <code>alpha</code> , which COSMOLATTICE already uses for the time-rescaling exponent
<code>k</code>	2	k	power at the minimum; must be an even integer ≥ 2 (checked, else the run stops with <code>RunParametersInconsistent</code>)
<code>kappa</code>	1	κ	plateau deformation
<code>g</code>	0	g	inflaton–daughter coupling
<code>initial_amplitudes</code>	required	φ_i, χ_i	GeV
<code>initial_momenta</code>	0 0	$\dot{\varphi}_i, \dot{\chi}_i$	GeV ²

Since k is a run-time number, powers are written as `pow(expr, k)` on field expressions. TempLat supports this, and an integer-valued floating exponent is well defined for negative bases, which occur for $x < 0$.

4.2 Program variables

COSMOLATTICE evolves $\tilde{\varphi} = \varphi/f_*$ in the time variable $d\tilde{\eta} = a^{-\alpha_t}\omega_* dt$. The program potential is $\tilde{V} = V/(f_*^2\omega_*^2)$. We chose:

$$f_* = M_P, \quad \omega_* = \sqrt{\lambda_k} M_P x_i^{(k-2)/2}, \quad \alpha_t = \frac{3(k-2)}{k+2}, \quad (3)$$

where $x_i = |\varphi_i|/M_P$ and λ_k is defined by the small-field limit $V \simeq (\lambda_k/k)M_P^{4-k}\varphi^k$:

$$\lambda_k^{(E)} = k \frac{3}{4} \lambda b^k, \quad \lambda_k^{(T)} = k \frac{3}{4} \lambda c^k. \quad (4)$$

Why these choices.

- $f_* = M_P$: the program field is $x = \varphi/M_P$, so the potential is written directly in terms of the functions of Sec. 3, with b, c unchanged. The plateau and the minimum are both $\mathcal{O}(1)$ in these units.

- ω_* is the oscillation frequency of a condensate of amplitude φ_i in the φ^k potential. For $k = 2$ it is the inflaton mass: $\omega_* = m_\varphi = \sqrt{\lambda/\alpha} M_P$ (E), $\frac{1}{2}\sqrt{\lambda/\alpha} M_P$ (T). For $k = 4$ it reduces to the `lphi4.h` choice $\omega_* = \sqrt{\lambda_4} \varphi_i$. One oscillation therefore takes $\Delta\tilde{\eta} \sim 2\pi$ at the start.
- α_t : in a φ^k potential the amplitude decays as $a^{-6/(k+2)}$, so the oscillation frequency decays as $a^{-3(k-2)/(k+2)}$. With $\alpha_t = 3(k-2)/(k+2)$ the frequency stays constant in program time. This gives $\alpha_t = 0$ for $k = 2$ (as in `tanh2.h`) and $\alpha_t = 1$ for $k = 4$ (as in `lphi4.h`); $\alpha_t = 3/2$ for $k = 6$, $\alpha_t = 2$ for $k = 10$.

With these choices, the code computes two constants once in the constructor:

$$\text{pref} = \frac{3}{4}\lambda \frac{M_P^2}{\omega_*^2}, \quad \text{qInt} = \frac{g^2 M_P^2}{\omega_*^2}, \quad (5)$$

so that $\tilde{V} = \text{pref} \cdot F(x) + \frac{1}{2} \text{qInt} x^2 \tilde{\chi}^2$, with $F = V/V_0$ the bracket expressions of Eqs. (1)–(2).

4.3 Numerically safe forms

Near the minimum $\kappa(1 - \cosh bx)$ and $(1 - \kappa)(\cosh bx - 1)$ suffer from cancellation. The code uses the identity $\cosh y - 1 = 2 \sinh^2(y/2)$:

$$G \equiv \kappa(1 - \cosh bx) + \sinh bx = \sinh bx - 2\kappa \sinh^2(bx/2), \quad (6)$$

$$D \equiv \frac{1}{2}(1 + \kappa + (1 - \kappa) \cosh bx) = 1 + (1 - \kappa) \sinh^2(bx/2). \quad (7)$$

The derivatives implemented are

$$\text{E: } F' = kG^{k-1}G', \quad F'' = k[(k-1)G^{k-2}G'^2 + G^{k-1}G''], \quad (8)$$

$$G' = b(\cosh bx - \kappa \sinh bx), \quad G'' = b^2(\sinh bx - \kappa \cosh bx), \quad (9)$$

$$\text{T: } F' = 2DD'T + D^2T', \quad F'' = 2(D'^2 + DD'')T + 4DD'T' + D^2T'', \quad (10)$$

with $D' = \frac{1}{2}(1 - \kappa)b \sinh bx$, $D'' = \frac{1}{2}(1 - \kappa)b^2 \cosh bx$, $T = \tanh^k(cx)$, $T' = kc \tanh^{k-1}(cx) \text{sech}^2(cx)$, $T'' = kc^2 \tanh^{k-2}(cx) \text{sech}^2(cx) [(k-1) \text{sech}^2(cx) - 2 \tanh^2(cx)]$. All of them were checked against finite differences of the potential. The second derivatives set the initial effective masses through `setInitialPotentialAndMassesFromPotential()`.

5 The initial-condition helper

`notes/attractor_ics.py` needs `numpy` and `scipy`. These are available in the `cosmolattice` conda environment.

Usage.

```
~/miniconda3/envs/cosmolattice/bin/python3 notes/attractor_ics.py E --alpha 1 --k 2 --
kappa 1 --nstar 50
```

It prints `lambda`, `alphaAtt`, `k`, `kappa`, `initial_amplitudes` and `initial_momenta` in the `.in` syntax, plus a comment line with the slow-roll n_s , r , x_* , x_{end} and H_{end} .

Method.

1. The background equation is integrated in e -folds N , where the trajectory $x(N)$ does not depend on λ :

$$x'' = -(3 - \varepsilon_H) \left(x' + \frac{F'(x)}{F(x)} \right), \quad \varepsilon_H = \frac{1}{2} x'^2. \quad (11)$$

It starts on the slow-roll attractor, $x' = -F'/F$, at a field value x_0 increased until the slow-roll estimate gives at least $N_* + 15$ e -folds.

2. The end of inflation is the event $\varepsilon_H = 1$. This gives x_{end} and x'_{end} .
3. x_* is the value N_* e -folds before the end, read from the dense ODE solution.
4. λ is fixed from $A_s = V_*/(24\pi^2\varepsilon_{V*}M_P^4)$ with $A_s = 2.1 \times 10^{-9}$, i.e. $\lambda = 32\pi^2 A_s \varepsilon_{V*}/F(x_*)$.
5. The initial conditions are set at the end of inflation:

$$\varphi_i = x_{\text{end}} M_P, \quad \dot{\varphi}_i = M_P H_{\text{end}} x'_{\text{end}}, \quad H_{\text{end}}^2 = \frac{V(x_{\text{end}})}{M_P^2(3 - \varepsilon_H)}. \quad (12)$$

The ODE uses `rtol` = 10^{-10} . The quoted n_s , r are slow-roll estimates, accurate to $\sim 10^{-3}$ in n_s ; the paper's values come from solving the Mukhanov–Sasaki equation.

Check against the paper. For Starobinsky with $N_* = 55$ the helper gives $\lambda = 1.57 \times 10^{-10}$ (paper: 1.6×10^{-10}), $x_* = 5.32$ (paper: 5.35), $x_{\text{end}} = 0.615$ (paper's analytic estimate: 0.63), $n_s = 0.964$, $r = 0.0037$ (paper, exact: 0.9649, 0.00355).

6 Choice of the parameters

6.1 Model parameters (α, k, κ)

The benchmarks are the parameter points for which the paper quotes predictions or which it identifies as best fits (its Results section, collected in Table 2 of `inflation_models.tex`):

- reference points: Starobinsky ($k = 2$, $\alpha = 1$) and the $k = 2$ T-model;
- steeper minima, which raise N_* and n_s through a stiffer reheating equation of state $w = (k - 2)/(k + 2)$: $k = 4, 6$ (E), $k = 6, 10$ (T) at $\alpha = 1$, and $\alpha = 5$ for E with $k = 6, 10$, where the paper finds overlap with the ACT region;
- deformed plateaus: $\kappa = 0.9999$ for $k = 2$, $\kappa = 0.99999$ for $k = 4, 6$ (see below), and the deformed E-model $k = 6$, $\alpha = 4$ point.

The value of κ for $k = 4, 6$. The paper's text says $\kappa = 0.9999$ for every deformed benchmark. With that value the $k = 4, 6$ points give a blue tilt ($n_s = 0.99$ – 1.04), far from the quoted $n_s \simeq 0.966$ – 0.970 . With $\kappa = 0.99999$ the helper reproduces them: E $k = 4$: $n_s = 0.969$, $r = 0.0044$ (paper 0.970, 0.004); T $k = 4$: $n_s = 0.966$, $r = 0.0042$ (paper 0.966, 0.004); T $k = 6$: $n_s = 0.968$ (paper 0.969). The corresponding figure file in the arXiv source is also named `fulltplots_kappa0p99999.pdf`, so we used $\kappa = 0.99999$ for these three files. The deformed E $k = 6$, $\alpha = 4$ point does match $\kappa = 0.9999$ ($n_s = 0.973$, $r = 0.017$; paper 0.974, 0.016), so it keeps that value. The paper's $r \simeq 0.012$ for deformed T $k = 6$ is not reproduced for either κ (we get 0.004).

6.2 Number of e -folds N_*

N_* is an input of the helper and fixes λ and the slow-roll observables. It does not change the lattice initial conditions much: x_{end} does not depend on N_* , while λ and $\dot{\varphi}_i$ change at the 10% level. Values were taken from the paper for its conservative reheating window $10^2 \text{ GeV} \lesssim T_{\text{RH}} \lesssim 10^{10} \text{ GeV}$:

- $k = 2$: $N_* = 50$, the value the paper uses for reference; it corresponds to $T_{\text{RH}} \simeq 5 \times 10^7 \text{ GeV}$ for Starobinsky. From the paper's App. B formula $N_* \simeq 59.55 - \frac{1}{3} \ln N_* + \frac{1}{3} \ln(T_{\text{RH}}/M_P)$, $N_* \simeq 45.7$ – 51.8 across the window.
- $k = 4$: $N_* = 55.7$ (E), 55.8 (T). For $w = 1/3$, N_* does not depend on T_{RH} (paper, App. B).
- $k = 6$: $N_* = 57$ – 58 , the range quoted by the paper.

- $k = 10$: $N_* = 60$, middle of the paper’s 59–61.5.

For $k \geq 6$ the true N_* depends on the inflaton fragmentation temperature, which is exactly what the lattice runs can measure. These values should be revisited once that is known.

6.3 Lattice and run settings

All 14 files share the same settings, which are untuned starting points:

Key	Value	Reason
expansion	true	self-consistent FRW expansion; needed for reheating
evolver	VV2	second-order velocity Verlet, as in <code>tanh2.in</code>
N	64	modest grid for exploration; 32 was used for the tests
kIR	0.05	$H_{\text{end}}/\omega_* \simeq 0.24$ for Starobinsky (the measured initial Friedmann value in program units, $0.058 \simeq 0.24^2$), so the box contains modes from above to well below the horizon scale
dt	0.02	~ 300 steps per initial oscillation period (2π); much smaller than the lattice spacing $\delta x = 2\pi/(N k_{\text{IR}}) \simeq 2$
kCutOff	2	initial vacuum fluctuations only below $\tilde{k} = 2$, inside the lattice range ($\tilde{k}_{\text{max}} \simeq \sqrt{3} N k_{\text{IR}}/2 \simeq 2.8$)
tMax	200	~ 30 oscillations; extend for fragmentation studies
tOutput*	0.5 / 5 / 20	frequent / infrequent / rare measurements
g	0	self-resonance only; set $g > 0$ to study preheating into χ
withGWs	false	off for the first runs

Before production runs, check resolution by varying **N**, **kIR** and **dt**, and watch the Friedmann-constraint output.

6.4 Benchmark files

File (<code>attractor...in</code>)	α	k	κ	N_*	λ	φ_i [GeV]	$\dot{\varphi}_i$ [GeV ²]	(n_s, r) slow roll
E_k2_a1_starobinsky	1	2	1	50	1.89×10^{-10}	1.50×10^{18}	-2.79×10^{31}	0.9604, 0.0044
E_k4_a1	1	4	1	55.7	1.60×10^{-10}	2.68×10^{18}	-2.29×10^{31}	0.9639, 0.0038
E_k6_a1	1	6	1	57	1.56×10^{-10}	3.53×10^{18}	-2.14×10^{31}	0.9645, 0.0037
E_k6_a5	5	6	1	58	7.23×10^{-10}	5.16×10^{18}	-2.16×10^{31}	0.9653, 0.0155
E_k10_a5	5	10	1	60	7.09×10^{-10}	7.37×10^{18}	-1.83×10^{31}	0.9660, 0.0152
E_k2_a1_kappa0p9999	1	2	0.9999	50	2.61×10^{-10}	1.50×10^{18}	-3.28×10^{31}	0.9737, 0.0062
E_k4_a1_kappa0p99999	1	4	0.99999	55.7	1.86×10^{-10}	2.68×10^{18}	-2.46×10^{31}	0.9694, 0.0044
E_k6_a4_kappa0p9999	4	6	0.9999	57	7.54×10^{-10}	4.93×10^{18}	-2.51×10^{31}	0.9729, 0.0169
T_k2_a1	1	2	1	50	2.08×10^{-10}	2.04×10^{18}	-2.44×10^{31}	0.9591, 0.0049
T_k6_a1	1	6	1	58	1.55×10^{-10}	4.83×10^{18}	-1.92×10^{31}	0.9646, 0.0037
T_k10_a1	1	10	1	60	1.45×10^{-10}	6.30×10^{18}	-1.84×10^{31}	0.9658, 0.0034
T_k2_a1_kappa0p9999	1	2	0.9999	50	2.78×10^{-10}	2.04×10^{18}	-2.82×10^{31}	0.9714, 0.0066
T_k4_a1_kappa0p99999	1	4	0.99999	55.8	1.80×10^{-10}	3.71×10^{18}	-2.11×10^{31}	0.9659, 0.0042
T_k6_a1_kappa0p99999	1	6	0.99999	57	1.80×10^{-10}	4.83×10^{18}	-2.07×10^{31}	0.9681, 0.0042

Table 1: The 14 benchmark parameter files. All numbers come from `notes/attractor_ics.py`; the generating command is recorded in the header of each file.

7 Validation

1. **Derivatives:** F' and F'' of all four families were compared with finite differences at a generic point ($k = 6$, $\kappa = 0.93$); agreement to $\sim 10^{-8}$.
2. **Compilation:** both headers build against COSMOLATTICE 2.0 (TempLat v1.0.2) with the `cosmolattice` conda environment (cmake 4.2.3, FFTW3).
3. **Short runs** ($N = 32$, $\tilde{t} \leq 20$):

Parameter file	NaN	Friedmann rel. error at $\tilde{t} = 19.5$	Behaviour
<code>attractorE_k2_a1_starobinsky</code>	none	2.3×10^{-4}	damped oscillation around $x = 0$
<code>attractorT_k2_a1</code>	none	3.8×10^{-4}	damped oscillation
<code>attractorE_k6_a1</code>	none	4.8×10^{-6}	anharmonic, asymmetric oscillation
<code>attractorE_k6_a4_kappa0p9999</code>	none	1.1×10^{-5}	anharmonic oscillation
<code>attractorT_k6_a1_kappa0p9999</code>	none	2.6×10^{-5}	anharmonic, symmetric oscillation

The initial program momentum of the Starobinsky run, $\tilde{\pi} = -0.342$, agrees with the hand conversion $\dot{\varphi}_i/(f_*\omega_*)$.

Not yet done: comparison of `attractorT` ($k = 2$, $\kappa = 1$) with the existing `tanh2` model (mapping $M = \sqrt{6\alpha} M_P$, $\Lambda^4 = \frac{3}{2}\lambda M_P^4$); resolution studies; long runs.

8 How to build and run

```
export PATH=~/.miniconda3/envs/cosmolattice/bin:$PATH # provides cmake and FFTW
cd ~/cosmolattice
mkdir build_attractorE && cd build_attractorE
cmake .. -DMODEL=attractorE && make -j8
./attractorE input=./models/parameter-files/attractorE_k2_a1_starobinsky.in
# any key can be overridden on the command line, e.g. N=128 tMax=500 g=1e-4
```

For the T-model replace `attractorE` with `attractorT`. To make a new benchmark, run the helper with the desired (α, k, κ, N_*) and copy its output into a copy of one of the existing `.in` files.

9 Open points

- Fix N_* self-consistently: for $k \geq 6$ measure the fragmentation time and temperature on the lattice and feed them back into the paper's App. B expression for N_* , which depends on $\max[T_{\text{RH}}, T_{\text{frag}}]$.
- Choose the daughter coupling g (and possibly trilinear or fermionic decay channels) for the reheating study.
- For $k \geq 6$, after fragmentation the equation of state tends to $w = 1/3$; consider $\alpha_t = 1$ for long runs.
- Tune lattice settings (Sec. 6) with resolution tests.